

4º CONBRAPA

Congresso Brasileiro de Planejamento Experimental e Análise de Dados

Minicurso 2

Agente LLM com LangChain

Profa. Dra. Patrícia de Oliveira e Lucas



Rio de Janeiro, 20 de Outubro de 2026

Quem é Patrícia?

Professora
IFNMG - Campus Salinas

Graduada em Sistemas de Informação
Universidade Estadual de Montes Claros - UNIMONTES

Mestre em Engenharia Elétrica
Universidade Federal de Minas Gerais - UFMG

Doutorado em Engenharia Elétrica
Universidade Federal de Minas Gerais - UFMG

Projeto de Pesquisa, Desenvolvimento e Inovação em Parceria com o Instituto KUNUMI

kunuminst UFMG

FUTURELAB

Roteiro

Agentes inteligentes baseados em Large Language Model (LLM)
Configurações de saída dos LLM
Engenharia de Prompts
Geração Aumentada por Recuperação
LangChain e LangGraph

Agentes inteligentes

No campo da Inteligência Artificial (IA), um agente é uma entidade artificial projetada para perceber seu ambiente por meio de sensores, tomar decisões com base nessas percepções e executar ações de forma autônoma para alcançar seu(s) objetivo(s).

A autonomia, nesse contexto, refere-se à capacidade do agente de operar sem intervenção humana direta, selecionando e realizando ações adequadas diante de diversas situações que possam surgir.

Evolução

Agentes simbólicos.

Agentes reativos.

Agentes baseados em aprendizado por reforço.

Agentes baseados em aprendizado por transferência e meta-aprendizagem.

Agentes baseados em LLM.

Por que os LLM são adequados para a construção de agentes?

Autonomia: capacidade executar diversas tarefas sem a necessidade de instruções programadas explicitamente.

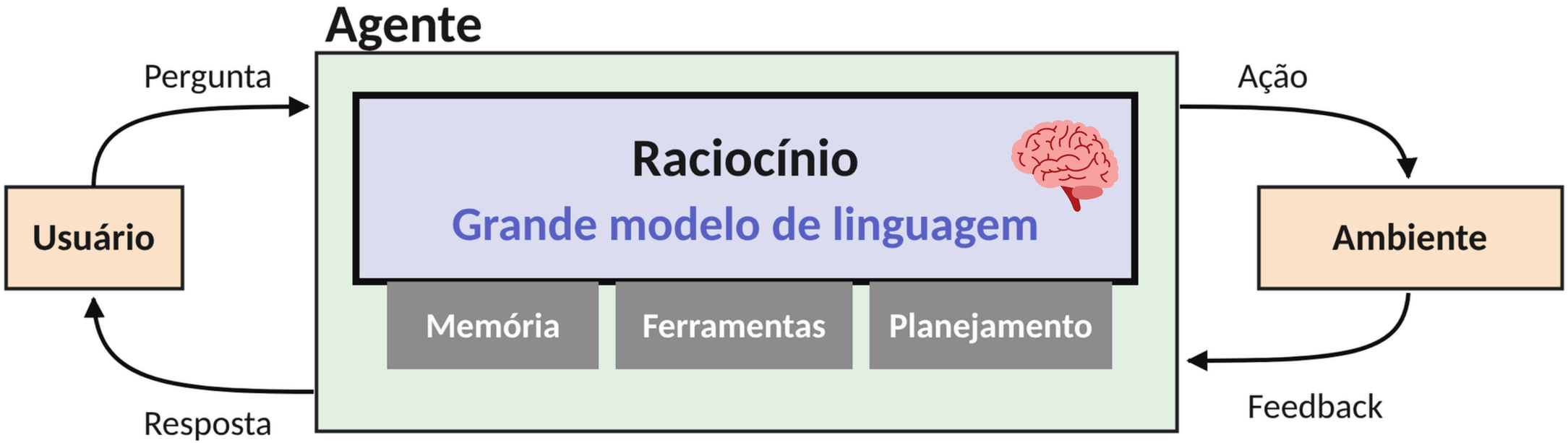
Agentes reativos: capacidade de responder a mudanças e estímulos. O espaço perceptivo dos LLM era restrito a entradas/saídas textuais, mas com os modelos multimodais é possível que os LLM recebam também informações visuais e auditivas.

Proatividade: capacidade de planejamento como reformular objetivos, decompor tarefas e ajustar os planos em resposta à mudanças do ambiente.

Habilidade social: capacidade de interagir com outros agentes (incluindo humanos). Melhorar o desempenho em tarefas por meio de comportamentos sociais como colaboração e competição.

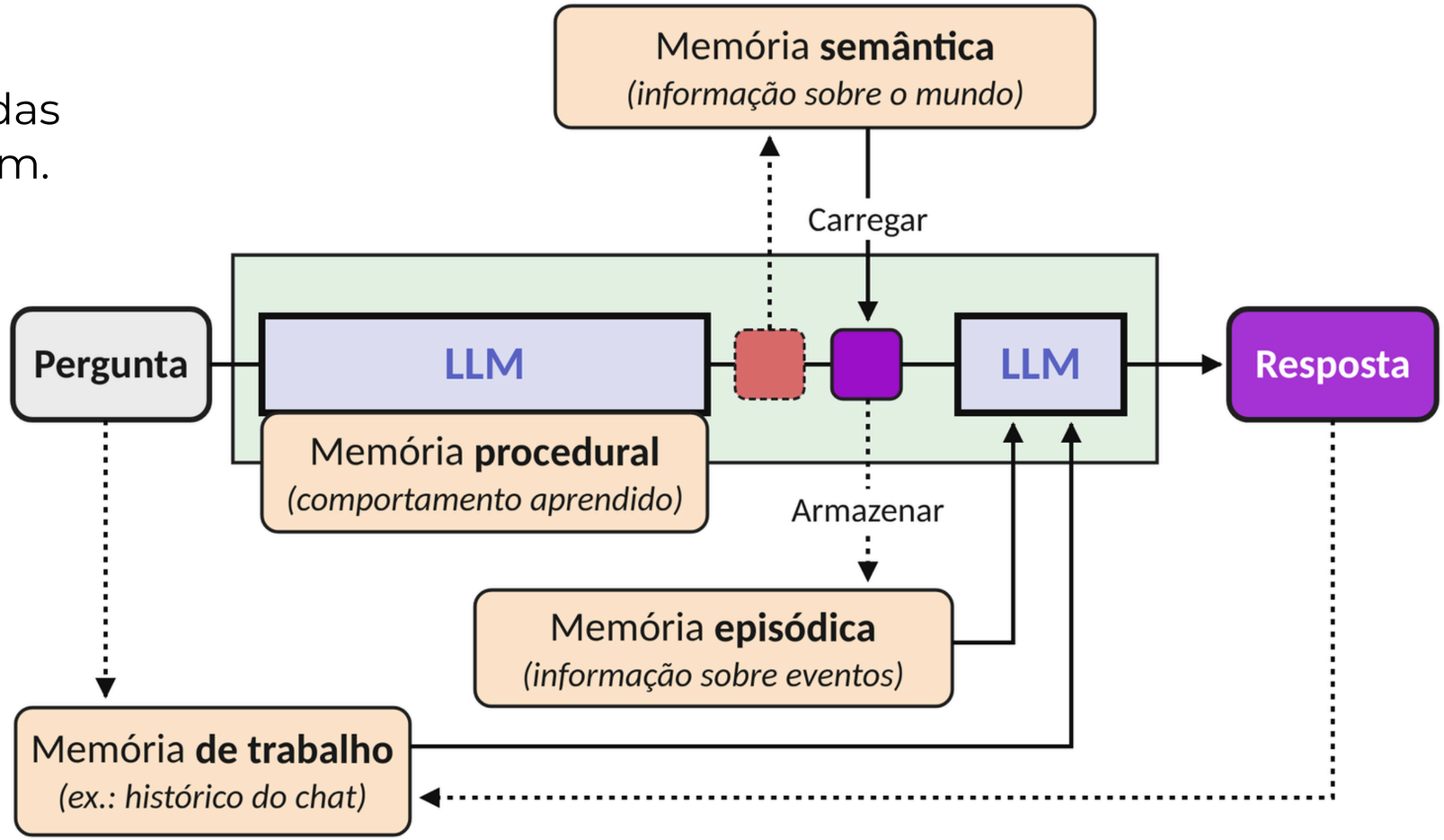
Uso de ferramentas: capacidade de executar funções, acessar APIs, etc.

Agentes baseados em LLM



Aumentando o LLM: memória

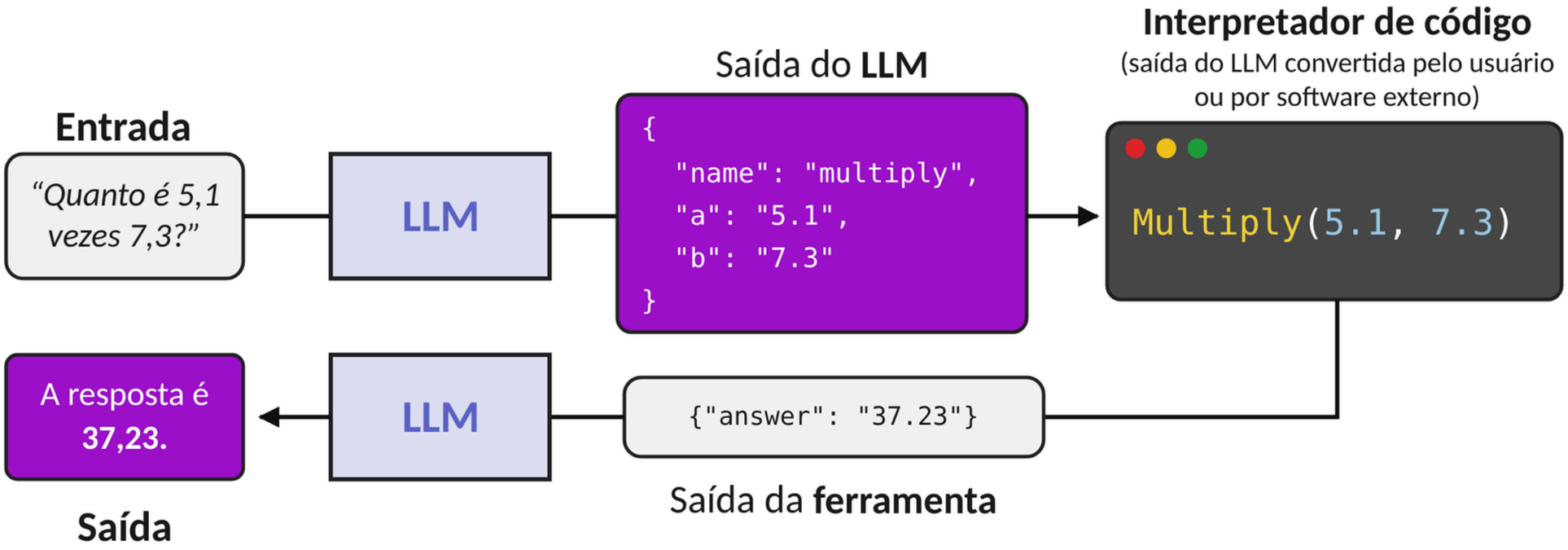
Sozinhos, os LLMs não se lembram de conversas passadas nem das ações que já tomaram.



Aumentando o LLM: ferramentas

Ferramentas externas ampliam a capacidade dos agentes.

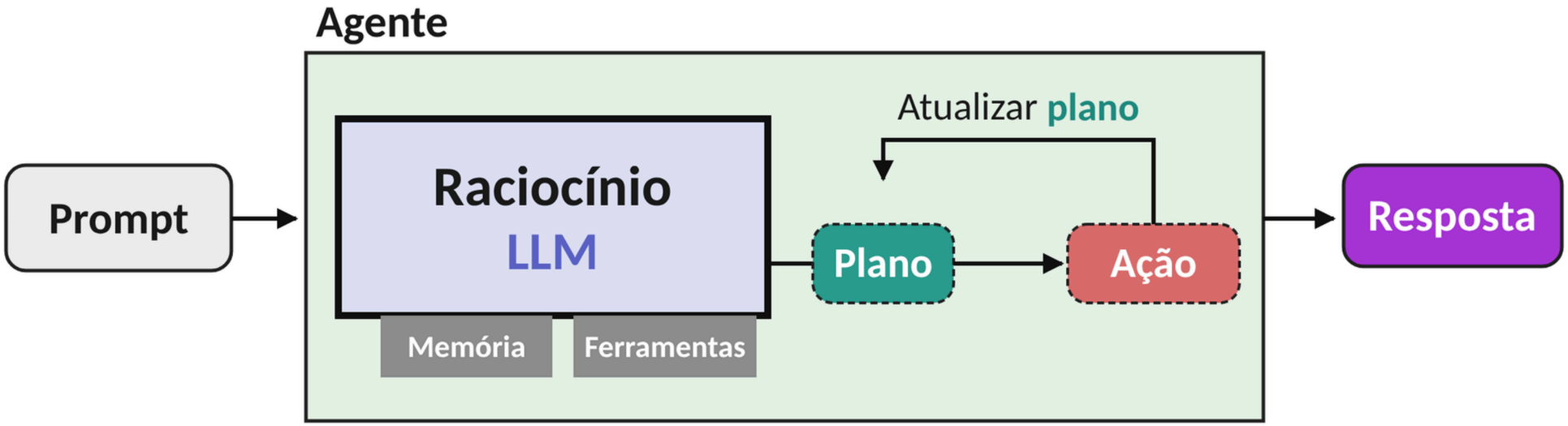
Porém, o LLM não executa ferramentas sozinho. Como função "texto entra, texto sai", ele só consegue expressar a intenção de agir.



Aumentando o LLM: planejamento

O ingrediente final para ir de um LLM "comum" a um agente é a capacidade de planejar e refletir.

O planejamento consiste em dividir uma tarefa grande em passos menores e executáveis.



Uso responsável

Agentes podem agir diretamente no mundo digital ou físico. Tudo depende do contexto.

Humano no circuito (human in the loop): quanto mais autônomo o agente, maior a necessidade de pessoas autorizarem, verificarem e auditarem suas decisões.

Guardrails: autonomia total pode ser exagerada e até prejudicial; guardrails direcionam o agente para os comportamentos esperados.

Desinformação: agentes continuam sendo LLMs e podem gerar informações incorretas com confiança (alucinações).

Configurações de saída dos LLM

Neste minicurso vamos analisar os LLMs como uma caixa preta, sob a perspectiva de usuários de um LLM pré-treinado.

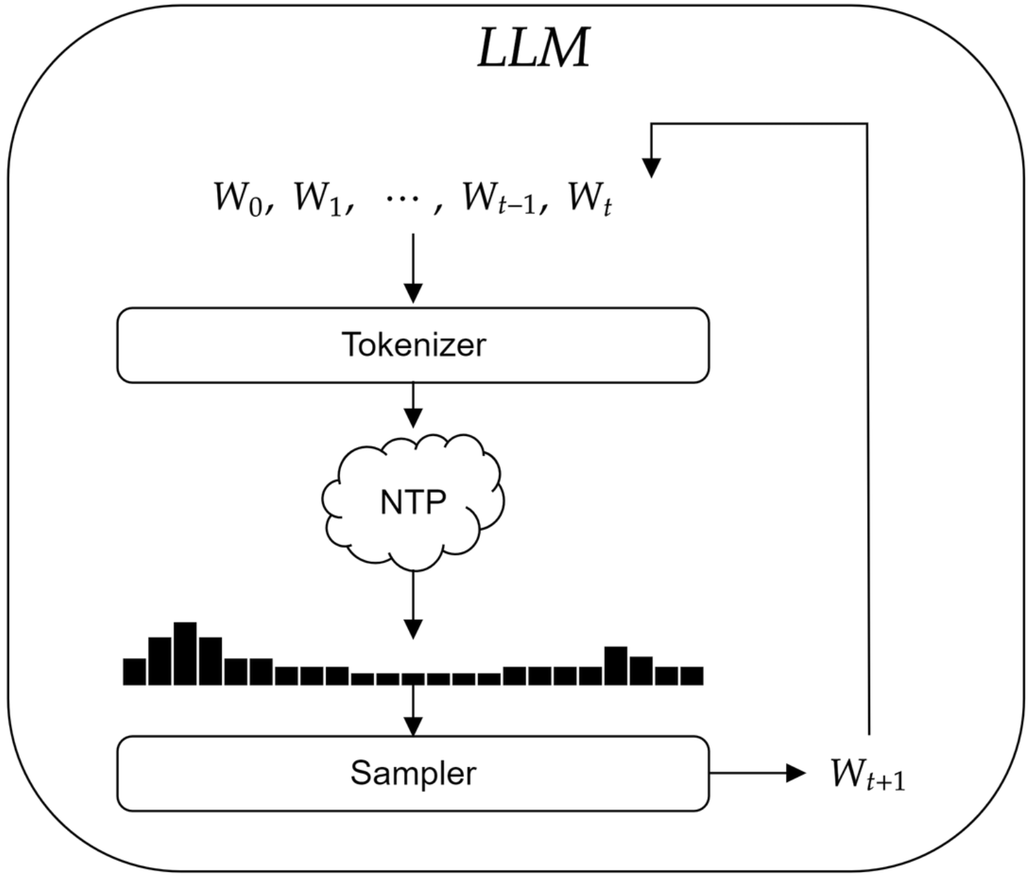
Os LLMs são modelos generativos de sequências:

$$W_{t+1}, \ \cdots, W_{t+N} \approx LLM(W_0, W_1, \ \cdots, W_{t-1}, W_t)$$

Na sua arquitetura interna contém um modelo NTP (Next Token Prediction):

$$W_{t+1} \approx NTP(W_0, W_1, \ \cdots, W_{t-1}, W_t)$$

Configurações de saída dos LLM



Amostragem não determinística: escolhe um token aleatoriamente do conjunto de tokens de maior probabilidade.

Gera diversidade linguística e maior criatividade generativa.

Aumenta as chances de alucinações.

Mas para agentes precisamos de determinísmo, ou seja, precisamos que os agente sejam previsíveis.

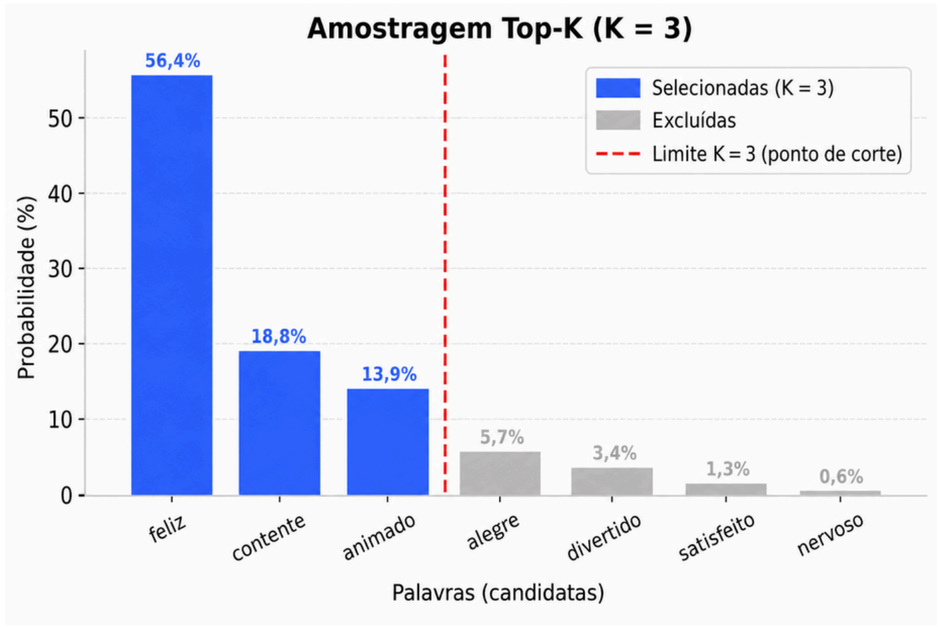
Parametros de geração:

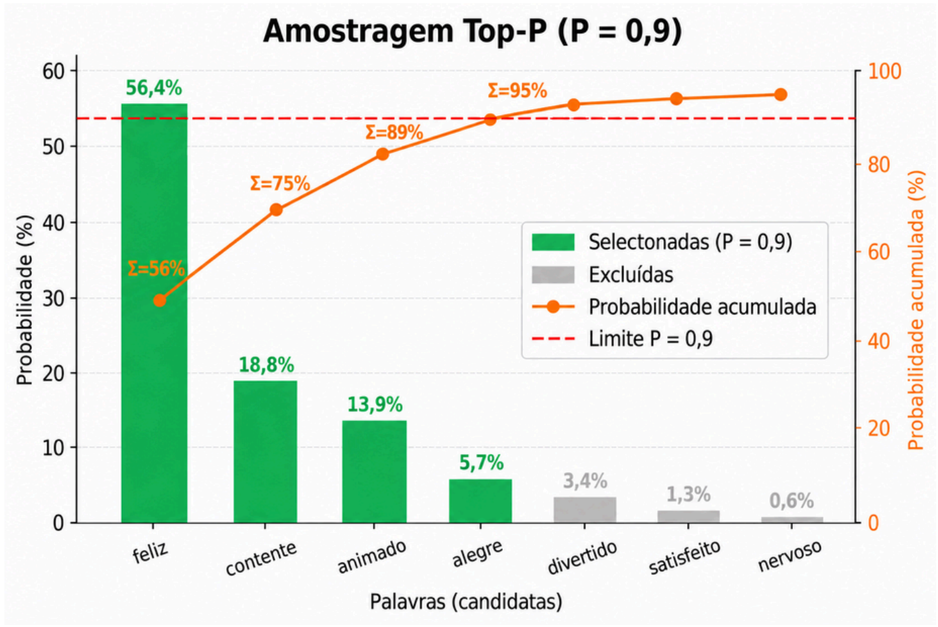
- ✓ Temperature: controla a aleatoriedade do processo generativo. Valor próximo de 0, mais determinísticas serão as respostas. Valor próximo de 1, maior aleatoriedade.

Configurações de saída dos LLM

Parametros de geração:

- ✓ Top-k: Restringe o número de tokens disponíveis limitando à k tokens de maior probabilidade.
- ✓ Top-p: Restringe o número de tokens disponíveis limitando à p de probabilidade acumulada.





- ✓ Max_tokens: número máximo de tokens gerados.

Referências

T. Masterman, S. Besen, M. Sawtell, and A. Chao, "**The Landscape of Emerging AI Agent Architectures for Reasoning, Planning, and Tool Calling: A Survey**," arXiv preprint arXiv:2404.11584, Apr. 2024.

T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "**Large Language Model Based Multi-agents: A Survey of Progress and Challenges**," in Proc. Int. Joint Conf. Artif. Intell. (IJCAI), Jul. 2024, pp. 8048–8057, doi: 10.24963/ijcai.2024/890.

Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Qin, Y. Zheng, X. Qiu, X. Huang, Q. Zhang, and T. Gui, "**The rise and potential of large language model based agents: a survey**," *Science China Information Sciences*, vol. 68, no. 2, Feb. 2025, doi: 10.1007/s11432-024-4222-0.

D. B. Acharya, K. Kuppan, and B. Divya, "**Agentic AI: Autonomous Intelligence for Complex Goals - A Comprehensive Survey**," IEEE Access, 2025, doi: 10.1109/ACCESS.2025.3532853.

GROOTENDORST, M.; ALAMMAR, J. **An Illustrated Guide to AI Agents.** Sebastopol: O'Reilly Media, 2026. ISBN 979-8341662698. Disponível em: https://www.oreilly.com/library/view/an-illustrated-guide/9798341662681/.

C. Shi, et al., "**A thorough examination of decoding methods in the era of LLMs**", arXiv preprint arXiv:2402.06925, 2024. [Online]. Available: https://arxiv.org/abs/2402.06925.

H. Wang and K. Shu, "**Make Every Token Count: A Systematic Survey on Decoding Methods for Foundation Models**," ResearchGate preprint, 2025. [Online]. Available: http://dx.doi.org/10.13140/RG.2.2.32726.36160.

Engenharia de Prompt

Um prompt é uma entrada para um modelo LLM, que é usado para orientar sua saída.

```
1 user_imput = "árvores"
2 template = f"Escreva um poema sobre o tópico: {user_imput}"
3 prompt = "Escreva um poema sobre o tópico: árvores"
```

Componentes de prompts: diretiva, exemplos, formatação da saída, instrução de estilo, persona e informações adicionais.

Técnicas de prompting: é um modelo que descreve como estruturar um prompt.

Engenharia de prompt: consiste no design estratégico de prompts para orientar a saída de LLMs sem a necessidade de alterar seus parâmetros internos. Essa abordagem é fundamental para adaptar os modelos a tarefas específicas, evitando processos custosos de retreinamento.

Engenharia de Prompt

Exemplos de técnicas de prompting:

Zero-shot: contém a descrição da tarefa, mas sem exemplos de como essa tarefa deve ser executada.

```
prompt = """

Traduza o texto para inglês: A previsão do tempo indica chuva amanhã.

"""
```

Few-shot: contém a descrição da tarefa e exemplos de como essa tarefa deve ser executada.

```
prompt = """

Traduza as frases abaixo para inglês:

"O gato está dormindo" -> "The cat is sleeping"
"Eu gosto de aprender" -> "I like to learn"
"A previsão do tempo indica chuva amanhã" ->

"""
```

Engenharia de Prompt

Exemplos de técnicas de prompting:

Chain-of-thought (CoT): induz o LLM a raciocinar de forma coerente e passo a passo.

```
template = f"""

P: Marcos tem duas cestas, cada uma contendo três bolas. Quantas bolas Marcos tem no total?
R: Uma cesta contém 3 bolas, então 2 cestas contêm 3 * 2 = 6 bolas.
P: {Pergunta}

"""
```

CoT few-shot

```
template = f"Vamos pensar passo a passo: {Pergunta}"
```

CoT zero-shot

Engenharia de Prompt

Exemplos de técnicas de prompting:

ReAct: combina raciocínio passo a passo com chamadas a ferramentas ou ações.

```
template = f"""

Responda às seguintes perguntas da melhor forma possível. Você tem acesso às seguintes ferramentas:

{tools}

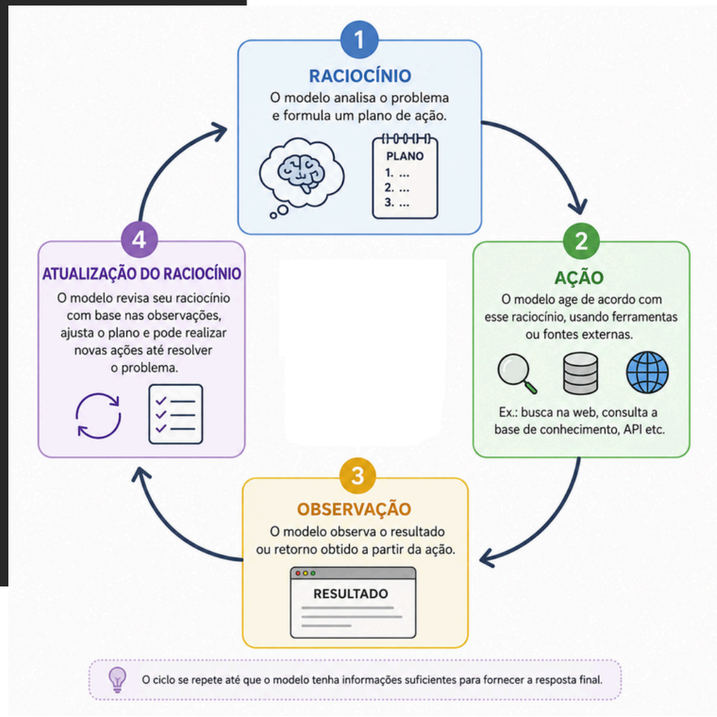
Use o seguinte formato:

Pergunta: a pergunta de entrada que você deve responder.
Pensamento: você deve sempre refletir sobre o que fazer.
Ação: a ação a ser tomada, deve ser uma dentre [{tool_names}].
Entrada da Ação: a entrada para a ação.
Observação: o resultado da ação
... (este bloco Pensamento/Ação/Entrada da Ação/Observação pode se repetir N vezes)
Pensamento: agora eu sei a resposta final.
Resposta Final: a resposta final para a pergunta de entrada original.

Comece!

Pergunta: {Pergunta}
Pensamento:{agent_scratchpad}

"""
```



Geração Aumentada por Recuperação

Retrieval Augmented Generation (RAG): tenta turbinar a geração de texto combinando a informação interna do LLM com informação externa armazenada em um banco de dados vetorial.

Vector store: banco de dados vetorial que faz o papel de memória de longo prazo para o agente.

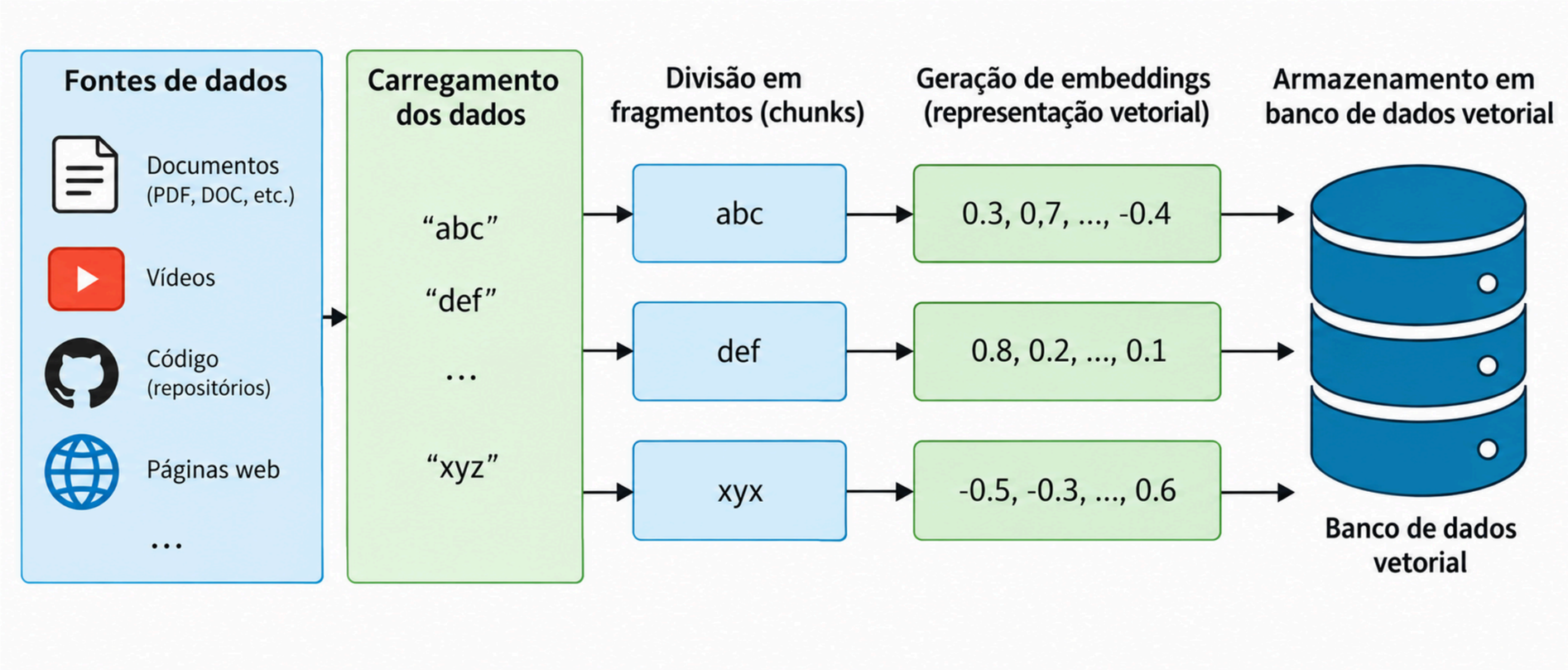
Armazena:

- ✓ Chunks: porções de texto.
- ✓ Embeddings: vetores de incorporação semântica que representam os chunks.
- ✓ Metadados: dados auxiliares.

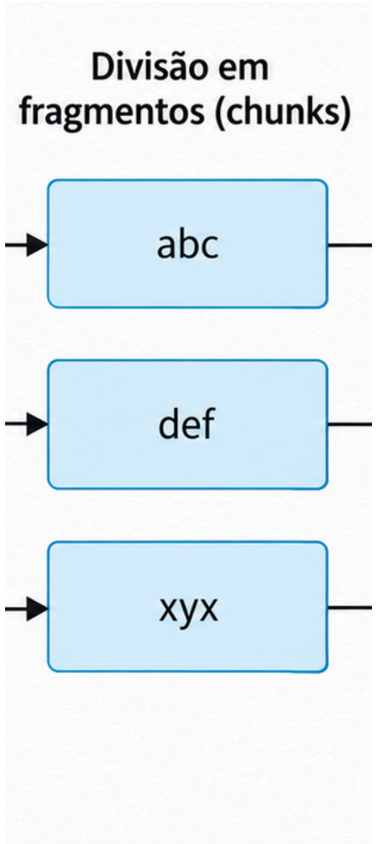
Uma aplicação RAG típica tem dois componentes principais: Indexação e Recuperação.

Geração Aumentada por Recuperação

Indexação: etapa em que os dados de uma ou mais fontes são coletados, processados e armazenados no vector store.



Geração Aumentada por Recuperação



Objetivo: particionar o texto em pedaços menores (chunks).

Chunks não podem ser grandes demais e nem pequeno demais.

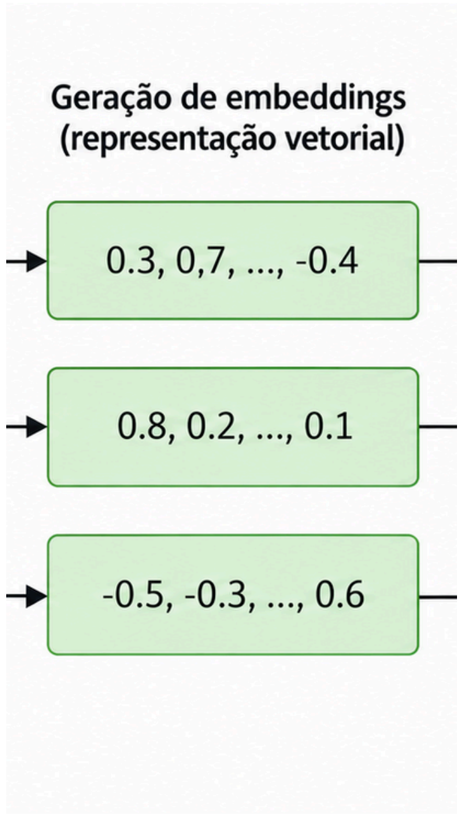
O ideal é que cada chunk seja um texto semanticamente coeso e autocontido.

Exemplos:

- Baseado em comprimento.
- Baseado na estrutura de documentos: HTML, Markdown, JSON, Código.
- Baseado na similaridade das frases.

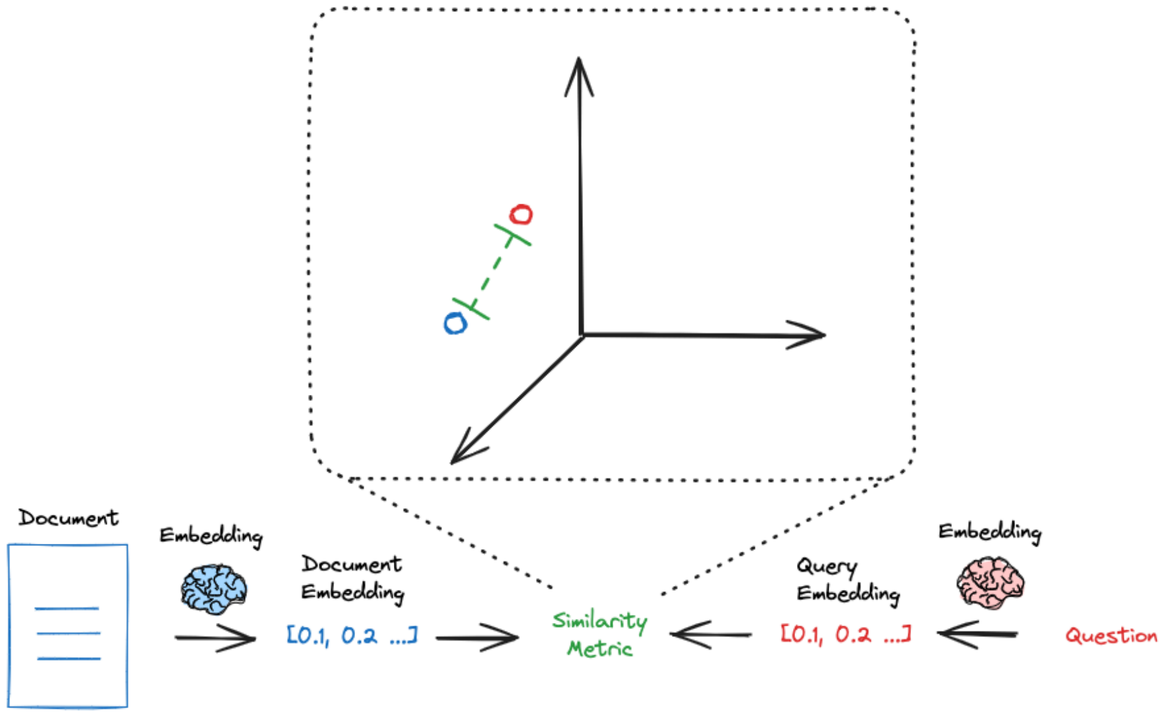
Docling

Geração Aumentada por Recuperação

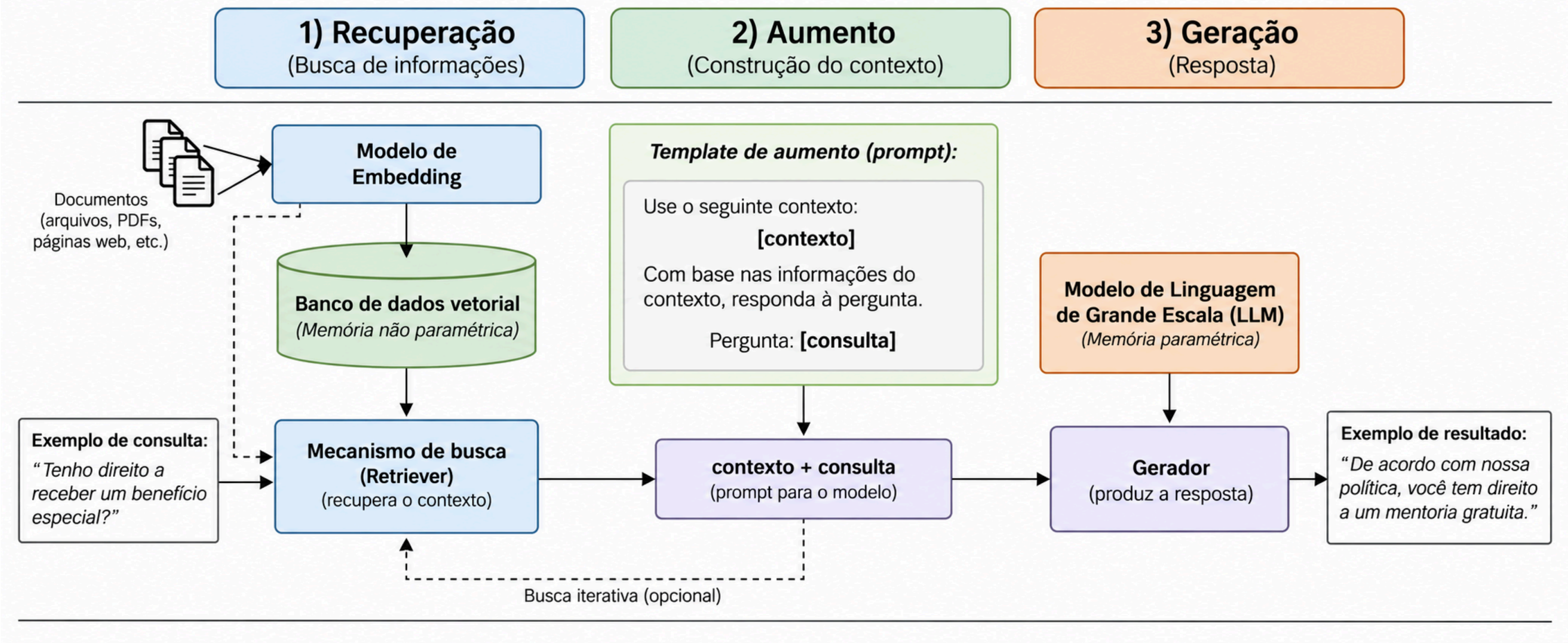


Embeddings são modelos.

Objetivo: incorporar a semântica de um chunk em um vetor numérico N-dimensional.



Geração Aumentada por Recuperação



Referências

P. Sahoo, et al., "**A systematic survey of prompt engineering in large language models: Techniques and applications**", arXiv preprint arXiv:2402.07927, 2024. [Online]. Available: https://arxiv.org/abs/2402.07927v2.

S. Schulhoff, et al., "**The prompt report: A systematic survey of prompting techniques**", arXiv preprint arXiv:2406.06608, vol. 5, 2024. [Online]. Available: https://arxiv.org/abs/2406.06608v6.

LangChain, "**Build a Retrieval Augmented Generation (RAG) App: Part 1**", LangChain Documentation, 2025. [Online]. Available: https://python.langchain.com/docs/tutorials/rag/.

N. Livathinos et al., "**Docling: An Efficient Open-Source Toolkit for AI-driven Document Conversion**", arXiv preprint arXiv:2501.17887, Jan. 2025. [Online]. Available: https://arxiv.org/abs/2501.17887.

MTEB, "**MTEB Leaderboard**", Hugging Face Spaces, 2025. [Online]. Available: https://huggingface.co/spaces/mteb/leaderboard.

GOLLNICK, B. **Generative AI with Python: The Developer's Guide to Pretrained LLMs, Vector Databases, Retrieval Augmented Generation, and Agentic Systems.** Boston: Rheinwerk Publishing, 2025. Capítulo 5: Vector Databases. ISBN 978-1-4932-2690-0.

LangChain

LangChain é uma framework para a construção de aplicações baseadas em LLM em python.

Ollama é uma plataforma de código aberto que permite executar e gerenciar Large Language Models (LLMs) localmente em seu próprio computador.







OBS: Os exemplos dos slides foram executados usando a API do DeepInfra, mas um notebook com exemplos rodando com Ollama será disponibilizado.

LangChain: Chat Models

Chat Models: modelos adaptados para conversação que possuem funcionalidades extras como integração com ferramentas e entrada e saídas estruturadas.

A entrada e a saída dos chat models consistem em uma lista de mensagens, que podem ser de um dos seguintes tipos:

- ✓ SystemMessage: Representa instruções de contexto ou regras gerais para o modelo. Serve para definir comportamento do modelo (estilo, linguagem, limitações).
- ✓ HumanMessage: representa a entrada do usuário humano. É o que o modelo deve interpretar como pergunta, comando ou dado de entrada.
- ✓ AIMessage: representa a resposta do modelo.